

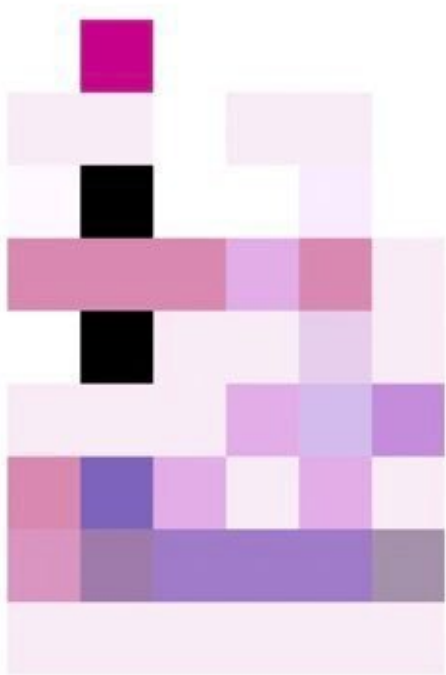
3>SHAKE

システムは「動く」だけでは
足りない

非機能要件・分散システム・トレードオフの基礎

2026/04/17 TECH BAR @NUTIC co-created with 3-shake

@nwiizo 20min



nwiizo

株式会社スリーシェイクでプロのソフトウェアエンジニアをやっているものです。SREやクラウドネイティブ技術を専門にしています。

趣味は読書、格闘技、グラビアです。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

about 3-shake



Mission

インフラをシンプルにしてイノベーションが起こりやすい世界を作る

Vision

労苦（Toil）を無くすサービスを適正な価格で提供し続ける

SRE



日本のSREをリードする
SRE総合支援からセキュリティ
対策を全方位支援

Data Engineering



**あらゆるサービスを
連携するハブになる**
クラウド型ETL/データパイプ
ラインサービスの決定版

Security



**事業者が抱える
セキュリティリスクを無くす**
ワンストップで実現する
セキュリティ対策

HR (Engineer Hiring)



**良いエンジニアに
良い案件を**
フリーランスエンジニアに
「今よりいい条件」を



今日お話しすること

1. 非機能要件とは何か — 「動く」の先にあるもの
2. 分散システムという選択肢 — なぜ分散し、何が難しいのか
3. トレードオフという現実 — 何を優先するかで形が変わる

ソフトウェアエンジニアが日々している「何を優先するか」の話を、できるだけ身近な例で紹介します。授業や本で出てくる言葉が、現場でどんな判断につながるのかをつかむのがゴールです。



この発表で解決できること

こんな疑問を持っていませんか？

- 「動けばOK」の先で何を考えるの？
- 分散すると何が増えるの？
- 何を見て設計を決めるの？

この発表で持ち帰れるもの

- 非機能要件という見方
- 分散すると増える難しさ
- 何を優先するかで設計が変わること

目標：「なぜその形にしたのか」を説明できるようになる

就職してから役に立つのは、ライブラリ名をたくさん知っていることより、何を優先したかを説明できることです。



今日の見方

今日は、用語を覚えるよりも、**1本の筋**で見てください。

1. 守るものが違う

同じ機能でも、速さ、正しさ、直しやすさのどれを優先するかで設計が変わります。

2. 分けると失敗が変わる

1台では起きない「成功したか不明」「途中まで成功」が出てきます。

3. 最後は判断になる

全部は取れないので、何を守るために何を引き受けるかを決めます。

この流れで、ソフトウェアエンジニアの仕事の中身を見ていきます



非機能要件とは何か

「動く」は当たり前。問題は「どう動くか」

機能要件と非機能要件の違い

ECサイトの「商品検索」を例に考えてみます。

機能要件 (What)

商品名で検索できる。カテゴリで絞り込める。検索結果を価格順に並べ替えられる。

→ システムが「何をするか」

非機能要件 (How)

検索結果を0.5秒以内に返す。同時に1万人が使ってもダウンしない。24時間365日稼働する。

→ システムが「どう動くか」

機能が同じでも、非機能要件でシステムの設計はまったく変わる



代表的な非機能要件

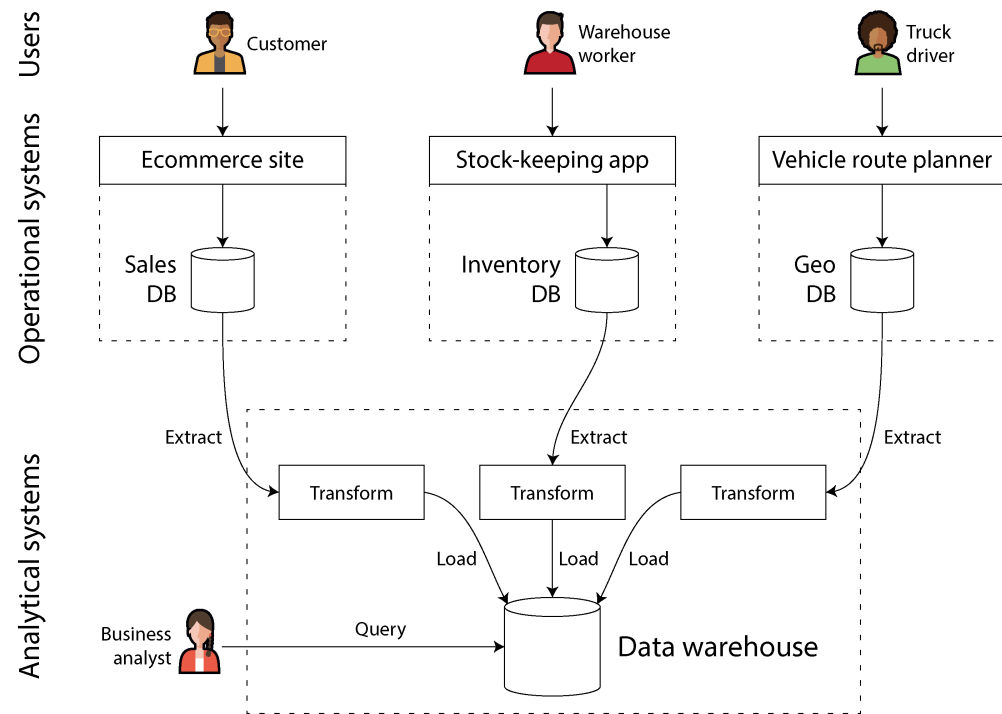
非機能要件	意味	身近な例
可用性	止まらずに動き続ける	LINEが大晦日でも使える
性能	速く応答する	Google検索が0.3秒で返る
スケーラビリティ	負荷増加に耐える	セール開始でもECが落ちない
耐障害性	壊れても復旧する	サーバー1台壊れても全体は動く
一貫性	データが矛盾しない	振込後の残高が正しい

ここで大事なのは、これらを**別々の単語として覚えないこと**です。実際のシステムでは、「速くしたい」「止めたくない」「ズレたくない」が同時に出てきます。だから設計では、いつも複数の性質をまとめて考えることになります。

設計の悩みは、1つの単語ではなく、複数の性質がぶつかるところから始まります



同じデータでも使い方が違えば欲しい性質も変わる



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 1-1 を引用

同じデータを扱っていても、日々の業務を回すシステムとあとから分析するシステムでは、重視するものが違います。

- 業務システム: すぐ返ること、止まらないこと、更新が正しいこと
- 分析システム: 大量データをまとめて読めること、履歴を持てること

同じデータでも、誰が何のために使うかで重視するものは変わります。だから設計では、まず何を優先するかを決める必要があります。

品質どうしは引っ張り合う

これらはそれぞれ別の話に見えますが、実際はつながっています。

たとえば「絶対に止めたくない」を優先すると、データの厳密さを少しゆるめることがある。

逆に「絶対にデータをズラしたくない」を優先すると、待ち時間が増えたり、止めざるをえないことがある。

この引っ張り合いが、あとで出てくるトレードオフです

品質は「困り方」で考える

学生のうちは、難しい分類名よりも「誰がどう困るのか」で見るほうがつかみやすいです。

遅い

検索や画面表示が遅いと、ユーザーは離れます。機能は合っていても使われません。

止まる

決済や申込が途中で止まると、その場で業務や体験が止まります。被害はすぐ見えます。

直せない

小さな変更でも時間がかかると、バグ修正も改善も遅れます。チームの速度が落ちます。

非機能要件は、こうした**困り方**をどこまで減らしたいかを決める話です。

品質は飾りではなく、「どんな困り方を減らしたいか」の宣言です



ここからは「設計の言葉」で見てみる

ここまでは、ユーザーや運用者が**どう困るか**の話として見てきました。

設計では、それを「あとで気をつけること」ではなく、**最初から構造を決める条件**として扱います。ここからは、その条件がどう設計に効くかを見ます。



「非機能要件」という名前の弱さ

私は、「非機能要件」という名前には少し弱いところがあると思っています。

「機能ではない」と聞こえる

主役ではない、あとで考えるものに見えやすい。

補足条件に見えやすい

速さや止まりにくさを、最後に足すもののよう誤解しやすい。

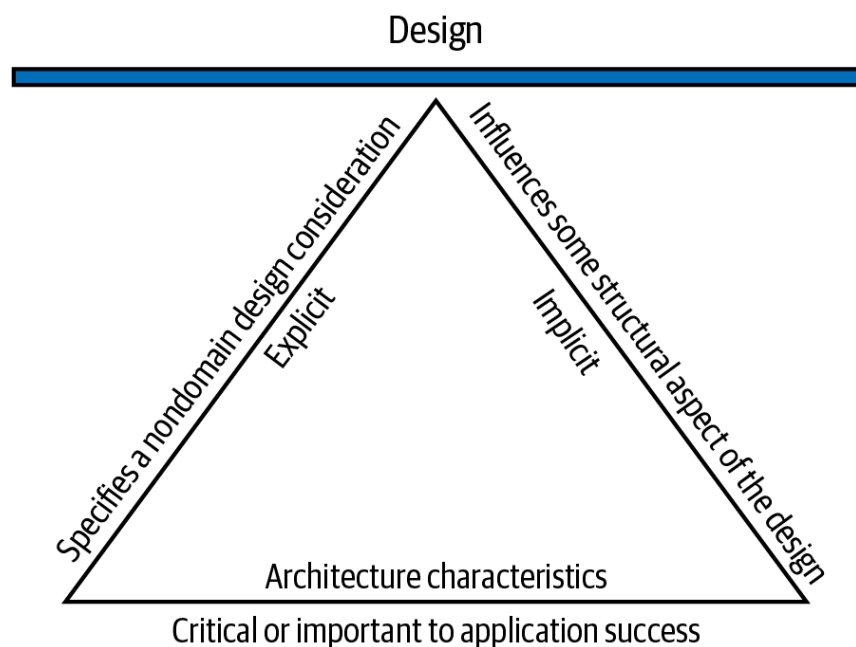
でも実際は形を決める

どんな構造にするかに直結する、かなり重要な条件です。

名前だけだと重要さが伝わりにくい。だから見方を変える必要があります



「非機能要件」ではなく「アーキテクチャ特性」と見る



ここでは、非機能要件を**アーキテクチャ特性**と呼びます。意味は、システムの形を決める大事な条件ということです。

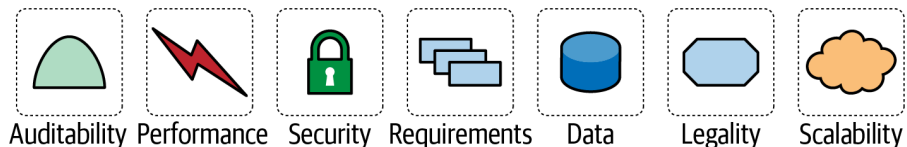
- ドメイン機能そのものではない
- 構造設計に影響する
- そのシステムの成功に重要である

可用性や性能は、実装の最後に足すものではなく、**最初に決めること**です。後回しにすると、実装が進むほど直しにくくなります。

出典: Fundamentals of Software Architecture, 2nd Edition, Figure 4-2 を引用

システムは「機能」だけでは設計できない

私は、ソフトウェアの解決策はドメイン要件とアーキテクチャ特性の両方でできていると考えています。



出典: Fundamentals of Software Architecture, 2nd Edition, Figure 4-1 を引用

- ドメイン要件: 商品を検索する、送金する、予約する
- アーキテクチャ特性: 速い、止まらない、変更しやすい、守られている

機能だけ見れば、ひとまず動くものは作れます。でも本番では、遅い、落ちる、直しにくい、という問題が出やすい。だから設計では、**何をするか**と**どう動いてほしいか**の両方を見ます。

同じ機能でも、守るものが違うと構造が変わる

ここで大事ななのは、非機能要件が「補足情報」ではなく、**構造を曲げる力**を持っていることです。

速く返したい

キャッシュ、検索インデックス、読み取り専用の複製を使いたくなります。

二重課金を避けたい

同じ依頼を見分けるIDや、強い整合性、慎重なやり直し制御が必要になります。

小さく直し続けたい

単純な構造、境界の明確さ、自動テストを優先したくなります。

つまり、設計は「何を作るか」だけでなく、**何を守りたいか**で大きく変わります。



理由が言えない設計は、あとで弱くなる

設計で強いのは、図がきれいなものではなく、なぜそうしたのかを説明できるものです。

- なぜ同期通信を選んだのか
- なぜここだけ強い一貫性が必要なのか
- なぜ今は分散しないのか

ここが曖昧だと、あとから入った人には「なんとなくそうなっている設計」に見えます。逆に理由が言えれば、変更するときにも判断をやり直せます。

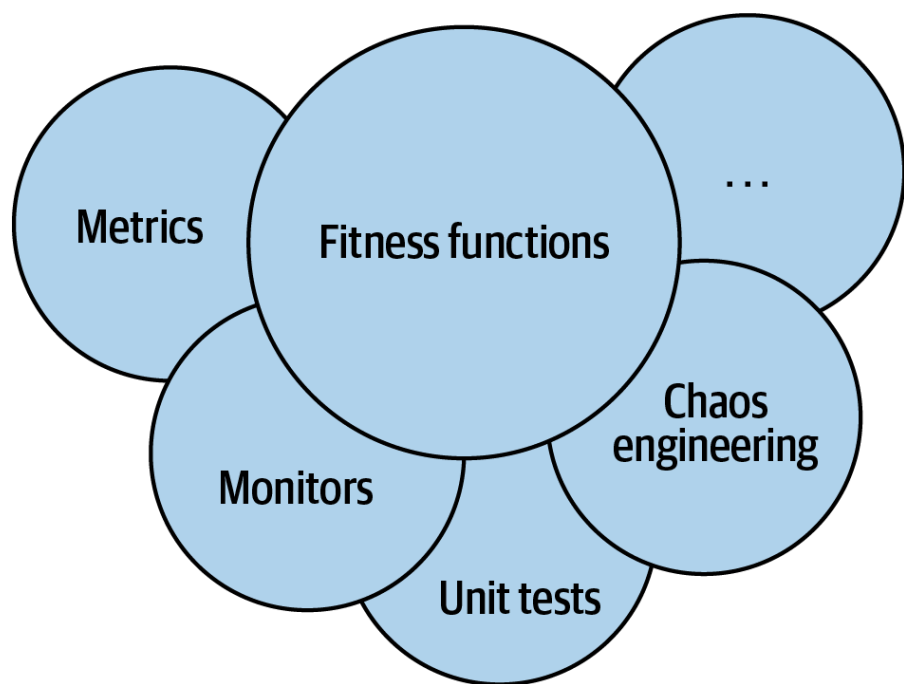
良い特性は「測れる言葉」に翻訳する

理由を言えるようにするには、「いい感じに速い」のような曖昧な言い方では足りません。

ふわっとした言葉	設計に使える言葉
高速にしたい	p95 応答時間 300ms (0.3秒) 以下
落ちにくくしたい	月のあいだ 99.95% 動いている
変更しやすくしたい	1機能を作り始めてから使えるまでを 1 日以内
安全にしたい	個人情報へのアクセスは、誰がいつ見たかの記録を必ず残す

平均だけ見ると、たまにもものすごく遅いケースが隠れてしまいます。だから p95 のように「100回のうち95回はここまでに返る」と見える指標が大事になります。

特性は「守る仕組み」まで含めて設計する



性能やレイヤ分離、依存関係の制約は、会議で言うだけでは守られません。だから私は、こうしたものを**設計ルールが守られているかを自動で確かめる仕組み**として継続的に検証するのが大事だと思っています。

- CI（変更を自動で確認する流れ）で循環依存を落とす
- 100回中95回の応答時間が基準を超えたら警告する
- 入口の処理から、データ保存の処理を直接呼ばせない

大事なのは、**良い設計を『みんな気をつけよう』だけで守らないこと**です。

出典: Fundamentals of Software Architecture, 2nd Edition, Figure 6-2 を引用

非機能要件を軽視するとどうなるか

ゲームのリリース日にサーバーが落ちる — スケーラビリティの見積もり不足。機能は完璧でも、ユーザーが殺到した瞬間に破綻する。発売日のSNSは阿鼻叫喚になる。

ECサイトの在庫が「あるのに買えない」 — 一貫性の問題。2人が同時に最後の1個をカートに入れた。片方は決済後に「在庫切れ」と表示される。

ページ表示が3秒かかるとユーザーの53%が離脱する — 性能の問題。機能的には正しく動いている。ただ遅いだけ。それだけでサービスは死ぬ。

設計は「何を作るか」より先に、「どんな事故を避けたいか」から始まる



ここまでで見えてきたこと

ここまででわかったこと

同じ機能でも、守りたいものが違えば、必要な構造も必要な工夫も変わります。

次の問い

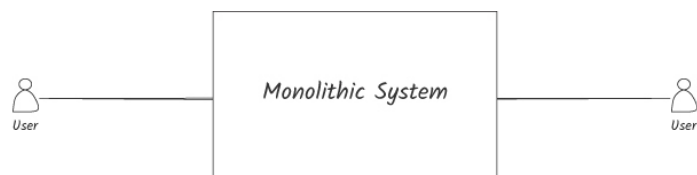
「では、その品質を満たすためにシステムを分けると、何が増えるのか？」
→ 分散システムの話へ



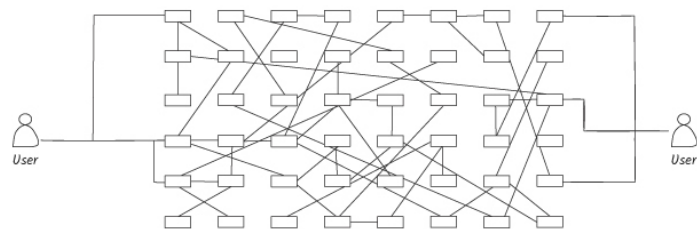
分散システムという選択肢

非機能要件を満たすために、システムを「分ける」

なぜシステムを分散させるのか



A



B

A: モノリス — 1台の箱にすべてが入っている。シンプルだが、CPUもメモリも物理的に上限がある。そしてその1台が壊れたら、すべてが止まる。

B: 分散システム — 複数のサービスがネットワークで繋がっている。見てわかるように、構造は一気に複雑になる。

分散の動機は3つ。**スケーラビリティ**（1台で足りなければ10台に）、**可用性**（1台壊れても残りが引き継ぐ）、**地理的分散**（ユーザーの近くにサーバーを置く）。

出典: Building Microservices, 2nd Edition, Figure 3.4 "Monolith vs Distributed" を引用

分散しないという選択も立派な設計

分散システムは強力ですが、常に正義ではありません。

最初はモノリスで十分なケース

- ユーザー数がまだ少ない
- チームが 3～5 人程度
- ドメイン理解がまだ固まっていない
- 速く試して学ぶことが重要

分散のコスト

- 通信失敗の処理が必要
- データ整合性が難しくなる
- 監視・デバッグが一気に複雑化
- チーム間調整が増える

「分散できるか」ではなく「分散する理由があるか」で決める



分散が生む新しい難しさ

サーバーを分けると、1台で動かしていたときには考えなくてよかった問題が一気に増えます。

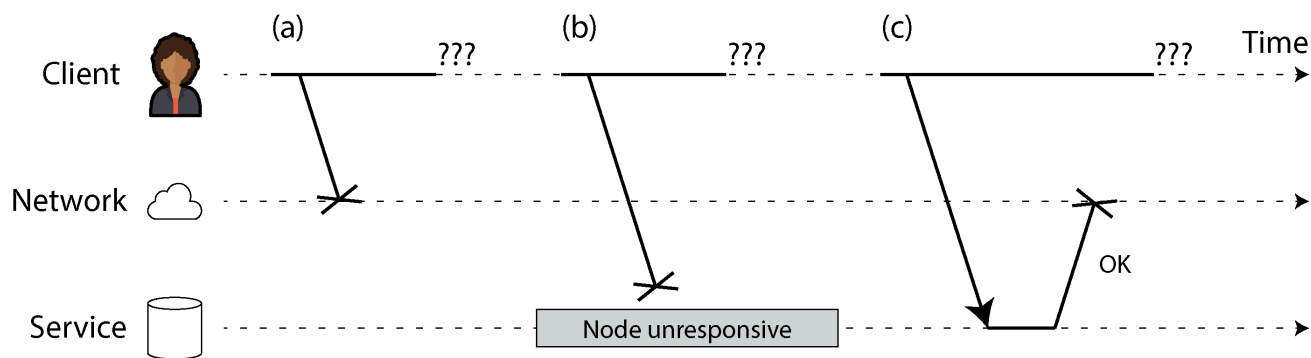
部分障害 (Partial Failure) — 全部が止まるわけではなく、「一部だけ壊れる」状態です。10台のうち2台だけおかしい、ある画面だけ失敗する、という形で現れます。分散すると「完全に動く / 完全に止まる」の二択ではなくなります。

データの整合性 — 同じデータを複数の場所に置くと、「どれが最新なの？」という問題が出ます。全員の足並みをそろえようとするが遅くなるし、急いで返そうとすると少し古い値を見ることがあります。

1台なら「**fault = failure**」。分散すると「**fault ≠ failure**」になる



ネットワークの不確実性



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 9-1 を引用

リクエストを送って返事が来ない。原因は3つ考えられる。

- (a) リクエストがネットワーク上で消えた
- (b) 相手のノードが落ちている
- (c) 処理は成功したがレスポンスが消えた

使う側から見ると、どれも同じ「返事が来ない」です。つまり、**本当に失敗したのか、成功したけど返事だけ消えたのか、見分けがつかない。**

この「不明」という第3の状態が分散システム最大の敵です。

失敗を前提にした基本戦略

ネットワークやサーバーの失敗は、「たまに起きる事故」ではなく、「いつか必ず起きるもの」として考えるべきです。

待ちすぎない (Timeout)

永遠に待たない。失敗を検知する。

やり直す (Retry)

一時的障害なら再試行する。

まずは「永遠に待たない」「一時的ならやり直す」という基本を入れます。ただし、ここで新しい問題が出ます。



やり直すなら「二重実行」を防がないといけない

やり直しを入れると、同じ処理が複数回送られる可能性があります。だから必要になるのが、何回送っても結果が壊れない性質（Idempotency）です。

たとえば「注文を作る」API が 2 回呼ばれても、注文が 2 件増えたら困る。
同じリクエストIDなら 1 回分として扱う、という工夫が必要になります。

失敗に強くするには、やり直せるだけでなく、やり直しても壊れないことが必要



「成功したか不明」への対処

注文 API を呼んで 3 秒後にタイムアウトしたとします。このとき、クライアントには 3 つの可能性があります。

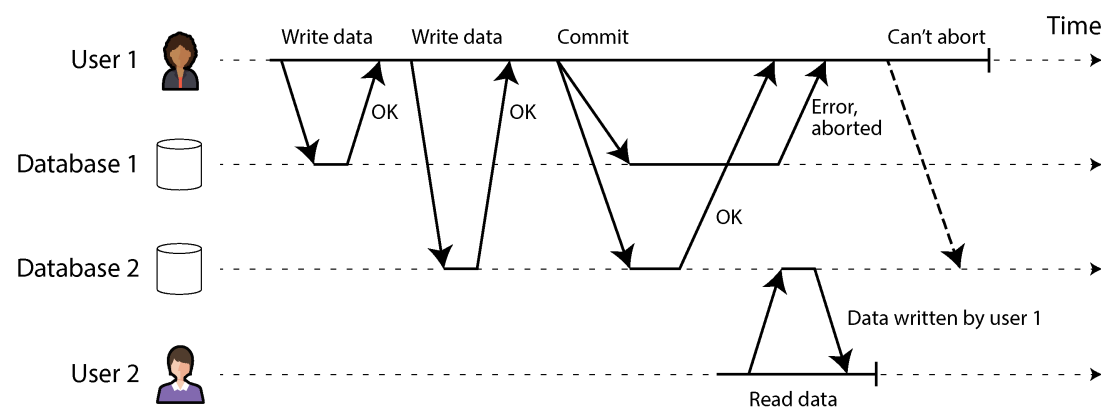
1. そもそも注文は作成されていない
2. 注文は作成されたが、応答だけ失われた
3. DB には書かれたが後続処理が途中で止まった

だから設計では、「その場ですぐ白黒つかない」ことを前提にします。注文番号やリクエストIDで**二重登録を防ぐ**、あとで状態を確認できるようにする、必要ならやり直しや取り消しを用意する。こういう地味な工夫が本番の強さになります。

派手さはありませんが、こういう「事故を未然に防ぐ仕組み」を考えるのも、プロのエンジニアの面白さです。



複数ノードでは「途中まで成功」が起こりうる



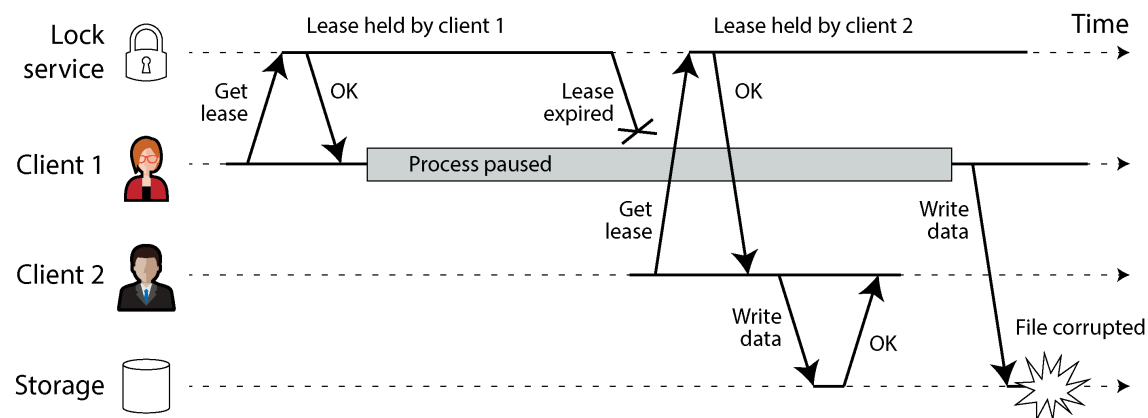
出典: Designing Data-Intensive Applications, 2nd Edition, Figure 8-12 を引用

1台の中で終わる処理なら、成功か失敗かを比較的是っきり扱えます。ところが複数ノードにまたがると、**片方では成功し、もう片方では失敗することが起こります。**

- あるDBには書けた
- 別のDBには書けなかった
- 利用者から見ると「どこまで成功したのかわからない」

ここが分散システムの難しさです。私は、分散とは「複数台に分けること」以上に、**中途半端な成功や中途半端な失敗と向き合うことだ**と思っています。

分散ロックは想像より危ない



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 9-4 を引用

「ロックを取ったからもう安全」と考えがちですが、ここには強い落とし穴があります。

- クライアント1がロック取得
- GC pause や stop-the-world で長時間停止
- その間に lease が失効
- クライアント2が新しいロック取得
- クライアント1が復帰して古い前提で書き込み

つまり、「ちゃんとロックしていたはず」という前提が崩れることがある。**分散システムでは、時間や実行順序を信じすぎないことが大事です。**

分散で厄介なのは「失敗」より「不明」

手元の関数呼び出しなら、たいていは「成功」か「失敗」です。分散ではそこに「成功したかどうかわからない」が入ります。

失敗

やり直す、待たない、別経路に切り替える、という対処が考えやすい。

不明

やり直すと二重実行になるかもしれない。状態確認や取り消しが必要になります。

分散で増えるのはエラーの数より、「状態が読めない場面」です



サービスを分けても、影響は残る

サービスを分けても、次の2つが強いと、実際には一緒に直すことになります。

いつも一緒に動かす必要がある

テスト、デプロイ、障害対応がいつもセットなら、見た目ほど独立していません。

相手の中身を知りすぎている

片方のDBや内部の都合をもう片方が前提にしていると、変更が連鎖します。

マイクロサービスに分けると、見た目は独立したように見えます。でも境界の切り方が雑だと、**別サービスなのに毎回セットで直す**ことになります。

分散は独立を約束しません。境界が悪いと、調整コストだけが増えます



分ける単位は「一緒に変わるもの」

サービスやモジュールは、「API担当」「DB担当」のような**技術名**で切るより、**変更理由がそろう単位**で切るほうが長持ちします。

- 技術で切る: 画面、API、DB が別々に増えて調整が増えやすい
- 責務で切る: 注文、決済、配送のように変更理由をそろえやすい

分散システムが難しいのは、台数が増えるからだけではなく、**分け方しだいで、変更のしやすさまで変わる**からです。



非同期にしても依存は消えない

イベント駆動やメッセージングを使うと、相手がその場で止まっても、こちらまで止まりにくくなります。

しかし、イベントに送信側の内部モデルがそのまま漏れていると、スキーマ変更のたびに受信側も巻き込まれます。

つまり、別々の場所で動いていても、相手の中身に強く依存していれば、扱いにくさは残ります。

非同期化は待ち合わせを減らしますが、意味の依存までは消しません



ここまでで見えてきたこと

ここまででわかったこと

分散システムは能力を増やしますが、その代わりに「不明」と「調整コスト」が増えます。

次の問い

「では、その増えた難しさの中で、何を守るために何を引き受けるのか？」
→ トレードオフの話へ



すべてはトレードオフ

全部を取ることはできない

大きい事故を小さい不便に変えるのが設計

CAP定理とその限界

分散システムでは、以下の3つを同時に完全には満たせないとされています。

C: 一貫性 (Consistency)

すべてのノードが同じデータを返す

A: 可用性 (Availability)

すべてのリクエストに応答する

P: 分断耐性 (Partition Tolerance)

ネットワーク障害でも動く

ネットワーク分断 (P) は現実にかかる。だからCとAのどちらを優先するかを選ぶことになります。ただし、CAP定理は「ネットワーク分断時」の話に限定されていて、返ってくるまでの時間や、1秒あたりにさばける量のような日常的なトレードオフはカバーしていない。実際の設計判断はCAPだけでは足りず、PACELC という「分断時だけでなく、平常時にも一貫性と速さの両立は難しい」と考える見方が必要です。



一貫性と可用性の選択

同じ「データの書き込み」でも、サービスの性質によって正解が変わります。

銀行の送金 → 一貫性を優先

AさんからBさんへの10万円の送金。途中でネットワーク障害が起きたとき、「とりあえず両方の口座から引いておく」は許されない。一時的にサービスが止まっても、残高は正確でなければならない。

SNSの「いいね」 → 可用性を優先

投稿への「いいね」数が一瞬だけ99と100でずれても誰も困らない。それより「いいね」ボタンが押せない方が問題。多少の不整合は後から修正すればいい。

止まってでも守るべきものと、少しずれても出し続けたいものは違います



同期通信と非同期通信のトレードオフ

同期通信

- 呼び出し結果がすぐわかる
- 実装は比較的わかりやすい
- 相手が落ちると自分も巻き込まれやすい

非同期通信

- 障害伝播を抑えやすい
- バッファリングしやすい
- 順序や重複の扱いが難しい

違いは速さの好みではありません。**その場で確定したいのか、あとでそろえばよいのか**の違いです。



非同期通信にも別の難しさがある

私の見方では、同期通信は**実行中の依存関係**を強くします。つまり、相手がその場で生きていないと自分も困りやすい。

ただし、非同期通信も万能薬ではありません。

イベントの順序が前後する、同じメッセージが重複する、送る側の都合が受け取る側に漏れる、といった別の難しさが出ます。

だから大事なのは「同期か非同期かの宗教」ではなく、**その場の確実さを取るのか、全体の止まりにくさを取るのか**です。



CAPだけでは足りない理由

CAP は有名ですが、これだけで実際の設計が全部決まるわけではありません。

観点	CAP が教えてくれること	それでも残る問い
分断時	C と A のどちらを優先するか	普段の遅さはどうする？
可用性	応答するかどうか	何秒で返すべきか？
一貫性	強いかわいいか	どの操作だけ強くするか？

現実の設計では、分断時だけでなく、普段の速さや運用の大変さまで含めて考えます。だから「平常時にどれだけ遅くなるか」や「どこまで品質を約束するか」まで見ないと、実際の設計判断にはつながりません。



アーキテクチャ特性は互いに干渉する

アーキテクチャ特性は1個ずつ別々に最適化できず、**お互いに影響し合う**と私は考えています。

セキュリティを上げる

暗号化、認可、監査が増え、性能や操作性が下がることもある

可用性を上げる

冗長化や非同期化が増え、一貫性や理解容易性が下がることもある

保守性を上げる

抽象化が増え、短期的な実装速度や局所性能が落ちることがある

しかも、どの特性を重視するかは組織ごとに違います。だからこそ、チーム内で共通言語を持ち、**測れる定義**に落とすことが大事です。



トレードオフは「何を守るために何を引き受けるか」

非機能要件の間には緊張関係があります。全部を同時に最大化できないからこそ、設計では守りたいもののために別の不便やコストを引き受けることになります。

トレードオフ	一方を追求すると	もう一方が
一貫性 ↔ 可用性	全ノード同期を待つ	レスポンスが遅くなる
性能 ↔ コスト	高速なハードウェアを使う	インフラ費用が膨らむ
シンプルさ ↔ 柔軟性	設定を少なくする	特殊なケースに対応できない
安全性 ↔ 利便性	認証を厳しくする	ユーザー体験が悪化する

設計は「大きい事故」を「小さい不便」に変える

良い設計は、痛みをゼロにすることではありません。致命傷になりうる事故を、受け入れられる不便に変えることです。

銀行

数秒止まる不便を受け入れて、誤送金という大事故を防ぐ。

SNS

数字が少しずれる不便を受け入れて、全体停止という大事故を避ける。

設計は「全部を良くする」より、「どの痛みを小さくするか」を決める仕事です



典型的な設計判断の比較

場面	重視するもの	よくある選択
銀行送金	正しさ、一貫性、監査性	同期確認、強い整合性、冪等キー
SNSタイムライン	可用性、応答速度	キャッシュ、非同期更新、結果整合性
ECカタログ	読み取り性能、検索性	検索インデックス、派生データ
管理画面	保守性、実装速度	単純なCRUD、過度な分散を避ける

ここで大事ななのは、「どれが唯一の正解か」ではなく、**何を守るために何を引き受けたのか**を説明できることです。表は答えを覚えるためではなく、**守るものが変わると選択も変わる**とつかむためのものです。



実務では「最小限の複雑さ」を選ぶ

設計は、「将来あるかもしれない全部の問題」に先回りして、最初から難しくするゲームではありません。

将来あるかもしれない負荷のために今から Kafka（大量のメッセージをやり取りする基盤）や、CQRS（書き込みと読み取りを分ける設計）と 20 マイクロサービスを入れるより、今の課題に対して**最小限で十分な複雑さ**を選ぶほうが良いことが多い。

複雑さそのものがコストです。理解するのも、壊れたときに直すのも、新しく入った人に教えるのも大変になります。だから「すごそうな設計」より「今の課題にちょうどいい設計」のほうが強いことが多いです。



トレードオフを判断する3つの問い

設計判断に迷ったとき、自分に問いかける3つの質問があります。

1. 壊れたとき、誰がいちばん困るか？

課金ミス、情報漏えい、数秒の遅さでは重さが違います。まず被害の大きさを見ます。

2. その痛みは、たまに起きるのか、毎回起きるのか？

年1回の障害と、毎日の遅さでは意味が違います。頻度で許容できるコストが変わります。

3. 大きい事故を、小さい不便に変えられているか？

一時停止で守れるなら止める。少し古い表示で済むなら速さを取る。痛みの置き場所を決めます。



この仕事の何が面白いのか

正解が固定されない

同じ技術でも、守るものが違えば答えが変わります。そこが面白い。

失敗のコストを読む

何が致命傷で、何が小さな不便で済むかを読み、構造に変えていきます。

判断が長く効く

決めた構造は、あとから入る人やユーザー体験、運用のしやすさにまで影響します。

コードを書くことはもちろん大事です。でも、それと同じくらい、**どの事故をどこで小さくするかを決めることが大事です。**

ソフトウェアエンジニアは、機能を作る人である前に、被害の形を設計する人です



まとめ

「動けばOK」の先で考えることは、どんな事故を避けたいかです。

分散すると増えるものは、サーバー台数より「不明」と「調整コスト」です。

設計を決める軸は、何を守りたいか、どれくらい起きるか、ユーザーにどう見えるかです。

この仕事のおもしろさは、状況に合わせて「大きい事故を小さい不便に変える判断」を組み立てるところにあります。

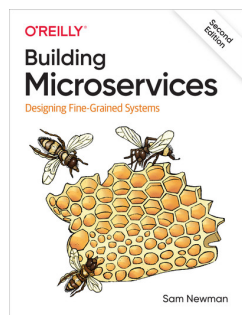
設計とは、何を守るために何を引き受けるかを決める仕事です



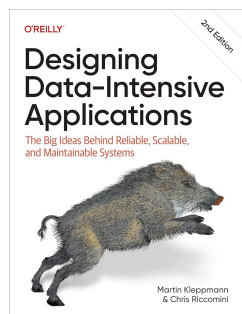
もっと深く学ぶための本



ソフトウェアアーキテクチャの基礎 (Mark Richards, Neal Ford) — 今日話した「非機能要件」を「アーキテクチャ特性」と呼び、体系的に分析する方法を解説。トレードオフ分析の章が秀逸。



Building Microservices, 2nd Edition (Sam Newman) — モノリスから分割する判断基準、サービス間通信、分散データの整合性など、分散システムの設計を実践的に解説。



Designing Data-Intensive Applications, 2nd Edition (Martin Kleppmann) — 今日の発表の骨格となった本。信頼性・スケーラビリティ・保守性の定義から、レプリケーション、CAP定理の批判的検討、分散合意まで。"There are no solutions. There are only trade-offs." を体現する一冊。

ありがとうございました

ご質問・ご相談はお気軽にどうぞ

@nwiizo | <https://3-shake.com>